Sommario

1. Struttura del dataset 2
2. build_dataset.py: creazione dell'oggetto Data 2
3. social_network.py: classe del dataset 2
4. master_loader.py: registrazione del dataset 3
5. social_network.yaml: configurazione del modello 3
6. Primo addestramento e metriche 3
7. Metriche per classe sul test 4
8. Correzione del numero di classi (main.py) 4
9. Variazione del numero di epoche 5
10. Ulteriori metriche: MAE, RMSE e classificazione binaria 5
11. Salvataggio del modello e inference 6
12. Riepilogo dei file nuovi e modificati 7

1. Struttura del dataset

Il dataset è composto da tre file, collocati in datasets/SocialNetwork/raw:

- real_social_network.edg: chi segue chi.
- real_posts.tsv: i post scritti dagli utenti.
- real_users_labels.tsv: la classe di ciascun utente (6 classi da 0 a 5).

Il problema è quindi una classificazione di nodi su un unico grafo. Dei post viene utilizzato solo il testo (content): le altre colonne del file sono annotazioni strettamente legate alla classe dell'utente e il loro utilizzo introdurrebbe data leakage.

2. build_dataset.py: creazione dell'oggetto Data

Lo script trasforma i file grezzi in un unico oggetto Data di PyTorch Geometric. Viene eseguito una sola volta, poiché la parte più costosa non deve essere ripetuta ad ogni addestramento.

- **Indicizzazione degli utenti**
  Gli ID testuali vengono mappati, tramite un dizionario, su indici interi da 0 a N-1, nell'ordine di prima apparizione. Gli archi sono salvati come coppie di indici nel tensore edge_index.
- **Pulizia degli archi**
  Con remove_self_loops e to_undirected vengono eliminati self-loop e archi duplicati e il grafo viene reso non orientato.
- **Le feature (x)**
  I post di ogni utente sono codificati con il modello pre-addestrato all-MiniLM-L6-v2, che produce un vettore di 384 valori per testo; per ogni utente si calcola la media dei vettori. Gli utenti senza post ricevono un vettore nullo.
- **Le etichette (y)**
  Ad ogni nodo è associata la sua classe; i nodi senza etichetta hanno valore -1.
- **Split**
  Solo i nodi etichettati vengono divisi in train (70%), validation (15%) e test (15%) con uno split stratificato e seed fisso. Il risultato sono tre maschere booleane.
- **Salvataggio**
  x, edge_index, y e le maschere vengono salvati in *datasets/SocialNetwork/processed_graph.pt*.

3. social_network.py: classe del dataset

GraphGPS richiede un dataset nel formato di PyTorch Geometric. Per questo è stata creata la classe SocialNetwork (graphgps/loader/dataset/social_network.py), sottoclasse di InMemoryDataset e analoga agli altri dataset del progetto. Al primo utilizzo verifica la presenza dei file grezzi, carica processed_graph.pt e ne salva una copia nella cartella

processed, che viene poi riutilizzata nelle esecuzioni successive. Se processed_graph.pt viene rigenerato, la cartella processed va eliminata.

4. master_loader.py: registrazione del dataset

master_loader.py seleziona il dataset da caricare in base alla configurazione.

Sono state apportate due modifiche: l'import della classe SocialNetwork e un nuovo caso SocialNetwork nella funzione load_dataset_master(), sul modello di quello esistente per Actor. In questo modo il formato PyG-SocialNetwork indicato nel file .yaml carica i dati da datasets/SocialNetwork.

5. social_network.yaml: configurazione del modello

Ogni esperimento in GraphGPS è descritto da un file .yaml. Come riferimento è stato scelto actor-GPS.yaml, poiché anche Actor è un problema di classificazione di nodi su un unico grafo con split fissati. La struttura e i parametri del modello sono gli stessi; sono stati modificati solo gli aspetti dipendenti dal dataset:

- **format: PyG-SocialNetwork** e **split_mode: standard**, per usare le maschere create in build_dataset.py;
- **node_encoder_name: LinearNode** al posto di LapPE: le positional encoding laplaciane richiedono una decomposizione su una matrice densa NxN, troppo onerosa in memoria per questo grafo;
- **layer_type: GCN+Performer** al posto di GCN+Transformer: il Performer approssima l'attention con costo lineare nel numero di nodi;
- **loss_fun: weighted_cross_entropy** e **metric_best: f1**, per tenere conto dello sbilanciamento tra le classi;
- **wandb disattivato** e **accelerator: cuda**.

6. Primo addestramento e metriche

L'addestramento si avvia con python main.py –cfg configs/GPS/social_network.yaml. Con il sampler full_batch l'intero grafo viene elaborato a ogni epoca; quindi, un'epoca corrisponde a un solo aggiornamento dei pesi e richiede meno di mezzo secondo su GPU. I risultati sono salvati in results/social_network.

stats.json riporta i valori per epoca, la cartella agg l'epoca migliore (quella con F1 più alto su validation). Le metriche calcolate sono loss, accuracy, F1 macro (media degli F1 delle singole classi) e AUC.

Con 200 epoche il modello ha ottenuto sul test un'accuracy di 0,481 e un F1 macro di 0,378. I valori di train, validation e test erano molto simili, indicando underfitting: il modello non apprendeva a sufficienza nemmeno sui dati di addestramento.

7. Metriche per classe sul test

Le metriche aggregate non descrivono il comportamento del modello sulle singole classi. È stato quindi aggiunto il calcolo del classification_report di scikit-learn (precision, recall, F1 e support per classe), solo sul test e solo per l'epoca migliore.

- **graphgps/logger.py**: nella funzione classification_multi(), se il logger è quello del test, la tabella viene calcolata e salvata nell'attributo last_report.
- **graphgps/train/custom_train.py**: in custom_train(), ogni volta che si trova una nuova epoca migliore, la tabella viene memorizzata; al termine viene scritta in *results/social_network/0/test/classification_report.txt*.

La tabella ha mostrato che il modello si comportava discretamente sulla classe 1 (circa il 40% degli utenti) ma molto peggio sulle classi meno numerose: la classe 4, ad esempio, aveva una recall di 0,15.

Classe	Precision	Recall	F1-score	Support
0	0,5086	0,3617	0,4228	575
1	0,5784	0,6799	0,6251	1481
2	0,3639	0,4461	0,4008	854
3	0,3437	0,2675	0,3009	456
4	0,3974	0,1505	0,2183	206
5	0,4167	0,2308	0,2970	130
Accuracy			0,4806	3702
Macro avg	0,4348	0,3561	0,3775	3702
Weighted avg	0,4734	0,4806	0,4678	3702

8. Correzione del numero di classi (main.py)

GraphGym determina il numero di classi contando i valori distinti di y, includendo anche il -1 dei nodi senza etichetta: il modello aveva quindi 7 uscite invece di 6. Ciò generava un avviso di torchmetrics e falsava l'AUC, calcolata anche su una classe inesistente. In main.py, tra create_loader() e create_model(), per il formato PyG-SocialNetwork viene ora impostato cfg.share.dim_out=6. Dopo la correzione lo strato finale è Linear(64, 6).

9. Variazione del numero di epoche

Per ridurre l'underfitting sono stati modificati max_epoch e num_warmup_epochs in social_network.yaml. I risultati sul test all'epoca migliore sono:

Addestramenti	Epoche / warmup	Epoca migliore	Accuracy	F1 macro
Primo	200 / 10	179	0,481	0,378
Secondo	1000 / 50	345	0,490	0,436
Terzo (attuale)	400 / 20	332	0,492	0,413

Con 1000 epoche l'F1 macro è aumentato, soprattutto grazie alle classi meno numerose. Tuttavia, dopo circa l'epoca 345 la loss sul validation ha ripreso a salire mentre quella di train continuava a scendere, segno di overfitting.

La configurazione attuale usa quindi 400 epoche e 20 di warmup.

10. Ulteriori metriche: MAE, RMSE e classificazione binaria

In logger.py sono state aggiunte altre metriche sul test, salvate nello stesso classification_report.txt:

- **MAE e RMSE**: poiché le classi rappresentano livelli ordinati, misurano di quanti livelli sbaglia il modello. Il MAE è la media degli errori assoluti, l'RMSE la radice della media degli errori al quadrato, che penalizza maggiormente gli errori grandi. Vale sempre RMSE ≥ MAE.
- **Classificazione binaria**: etichette vere e predette vengono trasformate in 0 (classi 0-2) e 1 (classi 3-5) e su di esse si calcola un secondo classification_report. Il modello non viene riaddestrato: cambia solo l'interpretazione delle sue predizioni.

Risultati del modello attuale (400 epoche con 20 di warmup, epoca migliore 332):

Classe	Precision	Recall	F1-score	Support
0	0,506	0,501	0,504	575
1	0,619	0,635	0,627	1481
2	0,372	0,445	0,405	854
3	0,347	0,281	0,310	456
4	0,373	0,185	0,247	206
5	0,407	0,369	0,387	130
Accuracy			0,492	3702
Macro avg	0,437	0,403	0,413	3702
Weighted avg	0,490	0,492	0,488	3702

MAE = 0,689, RMSE = 1,079. Una parte degli errori riguarda classi distanti due o più livelli.

Blocco	Precision	Recall	F1-score	Support
0 (classi 0-2)	0,866	0,926	0,895	2910
1 (classi 3-5)	0,635	0,472	0,542	792
Accuracy			0,829	3702
Macro avg	0,750	0,699	0,718	3702

L'accuracy binaria (0,83) è molto superiore a quella a 6 classi (0,49): molti errori avvengono tra classi vicine. Il blocco 3-5 però resta il punto debole del modello, con una recall di 0,472.

11. Salvataggio del modello e inference

Per valutare il modello senza riaddestrarlo sono state introdotte due modifiche:

- **social_network.yaml**: con *enable_ckpt: True* e *ckpt_best: True* viene salvato il modello dell'epoca migliore (*results/social_network/0/ckpt/332.ckpt*).
- **social_network-inference.yaml**: copia della configurazione con pretrained.dir: results/social_network, reset_prediction_head: False e train.mode: inference-only. Il modello salvato viene caricato e valutato una sola volta su train, validation e test.
- **custom_train.py**: anche la funzione inference_only() salva il classification_report.txt.

Il file di inference deve essere separato perché GraphGPS, all'avvio, cancella la cartella dei risultati con lo stesso nome del .yaml, che conterrebbe anche il checkpoint. La correttezza è stata verificata confrontando i risultati: validation e test coincidono con quelli dell'epoca 332 del training. In inference è possibile calcolare nuove metriche o valutare il modello su nuovi dati, mentre ogni modifica al modello o all'addestramento richiede un nuovo training.

12. Riepilogo dei file nuovi e modificati

File	Tipo	Contenuto / modifica
datasets/SocialNetwork/build_dataset.py	Nuovo	Costruzione dell'oggetto Data
graphgps/loader/dataset/social_network.py	Nuovo	Classe InMemoryDataset SocialNetwork
graphgps/loader/master_loader.py	Modificato	Import e caso "SocialNetwork"
graphgps/logger.py	Modificato	Report per classe, MAE, RMSE e report binario sul test
graphgps/train/custom_train.py	Modificato	Salvataggio del report (training e inference)
main.py	Modificato	6 uscite per PyG-SocialNetwork
configs/GPS/social_network.yaml	Nuovo	Configurazione dell'addestramento con checkpoint
configs/GPS/social_network-inference.yaml	Nuovo	Valutazione del modello salvato
.gitignore	Modificato	Esclusione dei dati, tranne build_dataset.py
README.md	Modificato	Descrizione del progetto e riferimenti